

Типы наиболее распространенных утечек памяти

Типы утечек при использовании внутреннего фреймворка:

- Каналы событий EventBus;
- Изменение значения, объявленного на уровне модуля.

Общие типы утечек в JS:

- Глобальные переменные;
- Ссылка на удаленный DOM-элемент;
- Замыкания функции.

Типы утечек при использовании внутреннего фреймворка

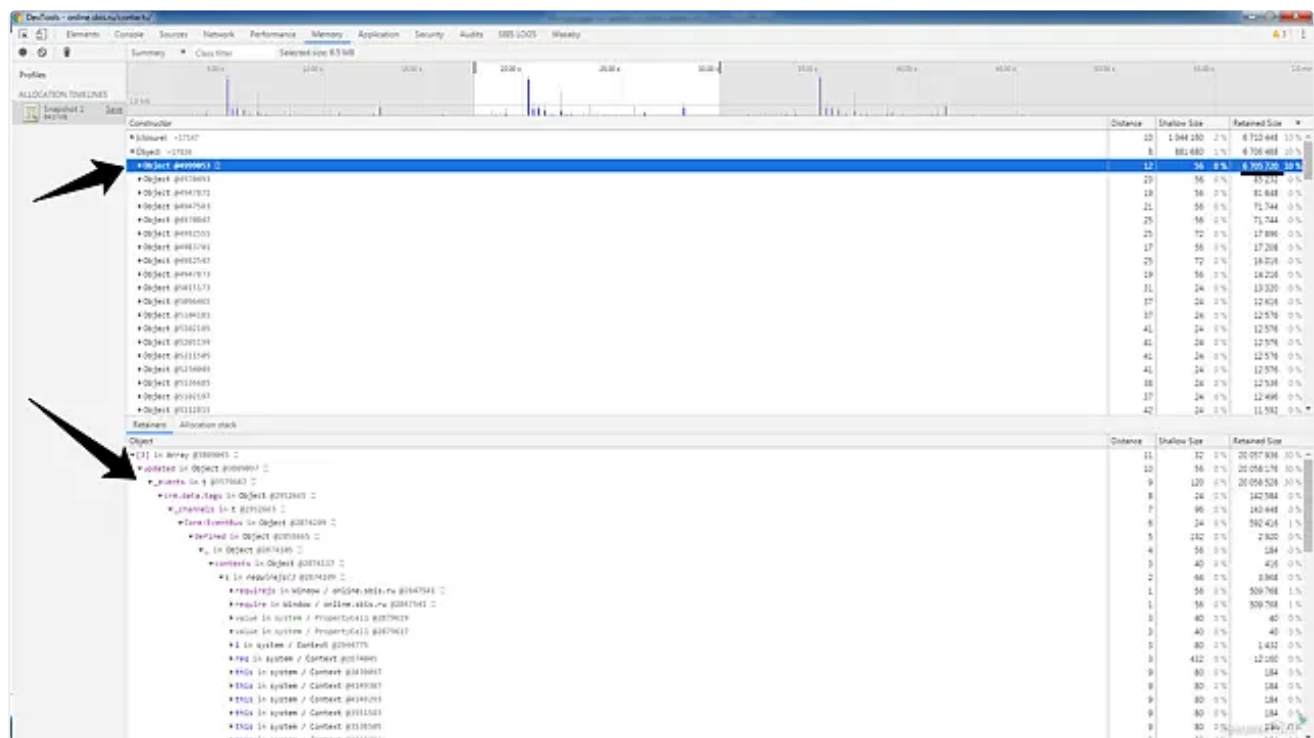
Каналы событий EventBus

Подписались на какой-либо канал событий и забыли отписаться от него.

Признаки

В цепочке владельцев должны быть следующие объекты:

- название шины, например, EventBus;
- channels;
- объект с названием канала;
- events;
- объект с названием события, на которое подписались.



Варианты решения

1. Подписка с автоотпиской при удалении контрола ([subscribeTo](#)). Данный способ работает только в WS3.

2. Для Wasaby-контролов нужно явно отписываться от канала, (например, [так](#)).

Изменение значения, объявленного на уровне модуля

Обычно такие проблемы связаны с непониманием двух вещей:

1. Сложные объекты передаются по ссылке.
2. Все переменные, объявленные на уровне модуля, инициализируются только один раз — при инициализации модуля.

Возьмем такой пример:

▼ Пример кода

```
1 const COMMON_ELEMENTS = [1, 2, 3];
2
3 export function getData(newData) {
4     const result = COMMON_ELEMENTS;
5     result.push(newData);
6     return result;
7 }
```

На первый взгляд, функция `getData` склеивает два массива и возвращает получившийся массив. Но на самом деле, `getData` попадет в массив `COMMON_ELEMENTS`.

На сервере модули инициализируются один раз для всех пользователей, и это значит, что массив `COMMON_ELEMENTS` будет держаться в памяти до остановки сервера. Т.е. таким нехитрым кодом мы получили две проблемы:

1. Утечку памяти.
2. Возможную утечку данных.

Исправить проблему легко, например, можно в теле функции сделать так:

▼ Пример кода

```
const result = COMMON_ELEMENTS.slice();
```

Неуникальные ключи в списках

При использовании одинаковых ключей возможны утечки памяти, связанные с перерисовкой. Также использование одинаковых ключей может ломать визуальное отображение (отрисовано меньше, чем должно, и некорректно работают действия над записями).

[Пример ошибки](#)

Общие типы утечек в JS

Глобальные переменные

Глобальные переменные в нашем случае могут быть источником утечки памяти. Если мы, например, постоянно пушим в глобальный массив без последующего удаления — это ведет к утечке.

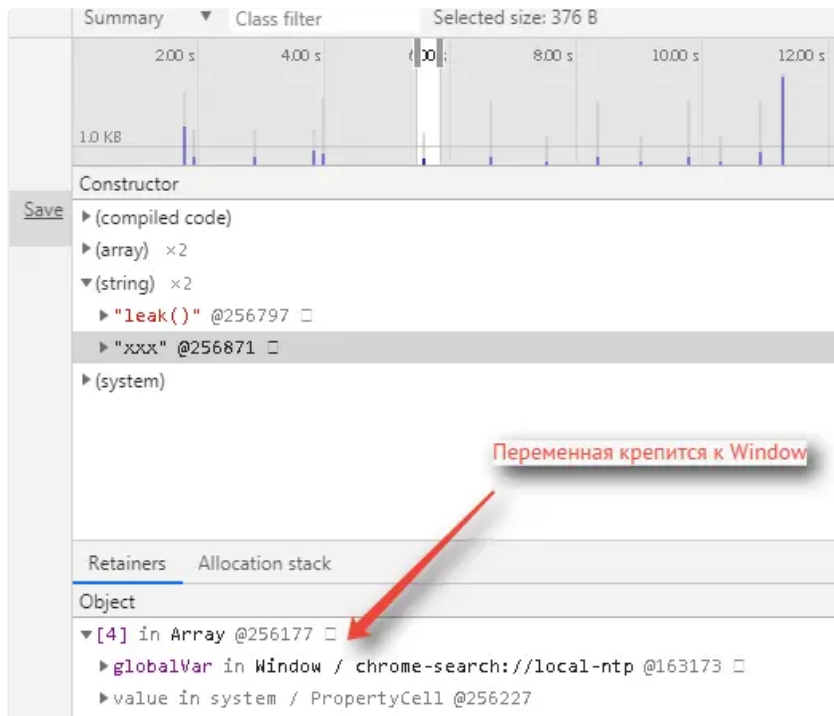
Признаки

У корня утечки должна быть глобальная переменная, которая крепится к `Window`.

```

> function foo(arg) {
  bar = "скрытая глобальная переменная";
}
< undefined
> window.bar
< undefined
> foo(1)
< undefined
> window.bar
< "скрытая глобальная переменная"
>

```



Решение

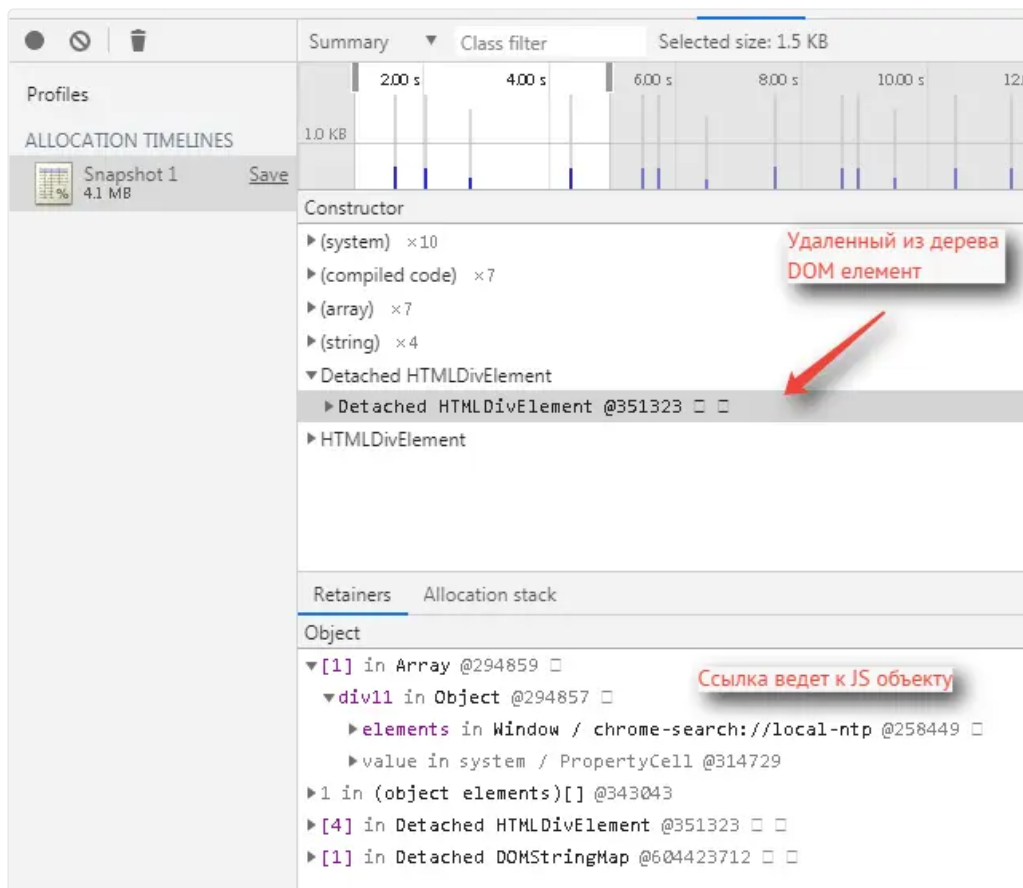
Не используйте глобальные переменные. Если есть необходимость, то строго следите за тем, чтобы она не пополнялась без очистки.

Ссылка на удаленный DOM элемент в коде

Если вы записываете в переменную ссылку на DOM-элемент, то после его удаления из дерева надо очистить также и ссылку из переменной.

Признаки

Если в коде есть ссылка на удаленный из DOM-дерева элемент, то он будет в куче помечен как Detached DOM element.



Решение

Надо удалить ссылку на удаленный из DOM-дерева элемент.

Замыкания функции

Утечка образуется из-за особенностей V8. Проще всего объяснить на примере:

1. При замыкании функции (unusedClosureFunction) в `[[Scopes]]` записываются все переменные из внешней функции, которые используются в unusedClosureFunction (это `outerLocalVar`). При этом `[[Scopes]]` будет общим для всех созданных в `mainFunc` функций, то есть в него входят сразу `outerLocalVar` и `outerLocalVar2`, несмотря на то, что принадлежат разным функциям.
2. Если хоть одна из внутренних функций будет жить, то весь `[[Scopes]]` не будет удален из памяти. Так, в примере в `mainFunc` в глобальной переменной `globalVar` создается функция `itsKeepClosureFunc`. Она остается доступной и после выполнения `mainFunc` и удерживает весь `[[Scopes]]`.
3. Далее при повторном вызове функции `mainFunc` переменная `outerLocalVar` уже имеет ссылку на функцию `itsKeepClosureFunc` и, соответственно, на предыдущий `[[Scopes]]`. Эта `outerLocalVar` попадает в новый `[[Scopes]]` при втором замыкании `unusedClosureFunction`.
4. Циклический вызов `mainFunc` сохраняет все созданные `[[Scopes]]`. Если говорить точнее, то в `[[Scopes]]` не сохраняются примитивы, так как у них немного другая логика выделения памяти, отличная от более сложных объектов.

▼ Пример кода

```

1  var globalVar = null;
2  var mainFunc = function () {
3      var outerLocalVar = globalVar;
4      var outerLocalVar2 = {num:'1'}
5      var unusedClosure = function unusedClosureFunction() {
6          if (outerLocalVar){}
7      };
8      var unusedClosure2 = function unusedClosureFunction2() {
9          if (outerLocalVar2){}
10     };
11     globalVar = {
12         longStr: new Array(1000000).join('1'),

```

```

13     otherClosure: function itsKeepClosureFunc() {}
14     };
15 };

```

Выглядит это все примерно так, как показано на рисунке

Текущая переменная

содержит себе созданную в предыдущем вызове функцию

с ее собственным Scope

В который также была записана предыдущая версия переменной outerLocalVar

Profiles

ALLOCATION TIMELINES

Snapshot 1 72 MB Save

Constructor

- Object x2
 - Object @2813217
 - Object @2813215
 - (closure)
 - system / Context
 - (concatenated string) x25
 - (compiled code) x3
 - (system) x4
 - (array) x3

Retainers Allocation stack

Object

- outerLocalVar in system / Context @2813449
 - context in itsKeepClosureFunc() @2813509
 - otherClosure in Object @2813453
 - outerLocalVar in system / Context @2813685
 - context in itsKeepClosureFunc() @2813745
 - otherClosure in Object @2813689
 - outerLocalVar in system / Context @2813921
 - context in itsKeepClosureFunc() @2813981
 - otherClosure in Object @2813925
 - globalVar in Window / chrome-search://local-ntp @2725909
 - value in system / PropertyCell @2753853

Решение

Если понять причину возникновения утечки, то разорвать ее будет не трудно. Надо, чтобы `[[Scopes]]` функции `itsKeepClosureFunc` не пересекался с `[[Scopes]]` `unusedClosureFunction`. Создайте

функцию `itsKeepClosureFunc` за пределами функции `mainFunc`, и в функции `mainFunc` просто ссылаетесь на нее.